In this case, the release fence ❷ has the same effect as if the store to `y` ❸ was tagged with `memory_order_release` rather than `memory_order_relaxed`. Likewise, the acquire fence ❺ makes it as if the load from `y` ❹ was tagged with `memory_order_acquire`. This is the general idea with fences: if an acquire operation sees the result of a store that takes place after a release fence, the fence synchronizes-with that acquire operation; and if a load that takes place before an acquire fence sees the result of a release operation, the release operation synchronizes-with the acquire fence. Of course, you can have fences on both sides, as in the example here, in which case if a load that takes place before the acquire fence sees a value written by a store that takes place after the release fence, the release fence synchronizes-with the acquire fence.

Although the fence synchronization depends on the values read or written by operations before or after the fence, it's important to note that the synchronization point is the fence itself. If you take `write_x_then_y` from listing 5.12 and move the write to `x` after the fence as follows, the condition in the assert is no longer guaranteed to be true, even though the write to `x` comes before the write to `y`:

```
void write_x_then_y()
{
    std::atomic_thread_fence(std::memory_order_release);
     x.store(true,std::memory_order_relaxed);
     y.store(true,std::memory_order_relaxed);
}
```

These two operations are no longer separated by the fence and so are no longer ordered. It's only when the fence comes *between* the store to `x` and the store to `y` that it imposes an ordering. Of course, the presence or absence of a fence doesn't affect any enforced orderings on happens-before relations that exist because of other atomic operations.

This example, and almost every other example so far in this chapter, is built entirely from variables with an atomic type. However, the real benefit to using atomic operations to enforce an ordering is that they can enforce an ordering on nonatomic operations and thus avoid the undefined behavior of a data race, as you saw back in listing 5.2.

5.3.6. Ordering nonatomic operations with atomics

If you replace `x` from listing 5.12 with an ordinary nonatomic `bool` (as in the following listing), the behavior is guaranteed to be the same.

Listing 5.13. Enforcing ordering on nonatomic operations